

web-值班邮件台

题目描述

值班邮件台

100

3638 solves

夜班值守，手别太快。别人的邮件别乱翻，自己的联调痕迹也别乱留。台子搭得仓促，收尾却没收干净，安静的界面下面，总还有点不该留下的东西。

题目地址：39.105.213.28:49103

Flag

提交

题目背景

核心漏洞链条：

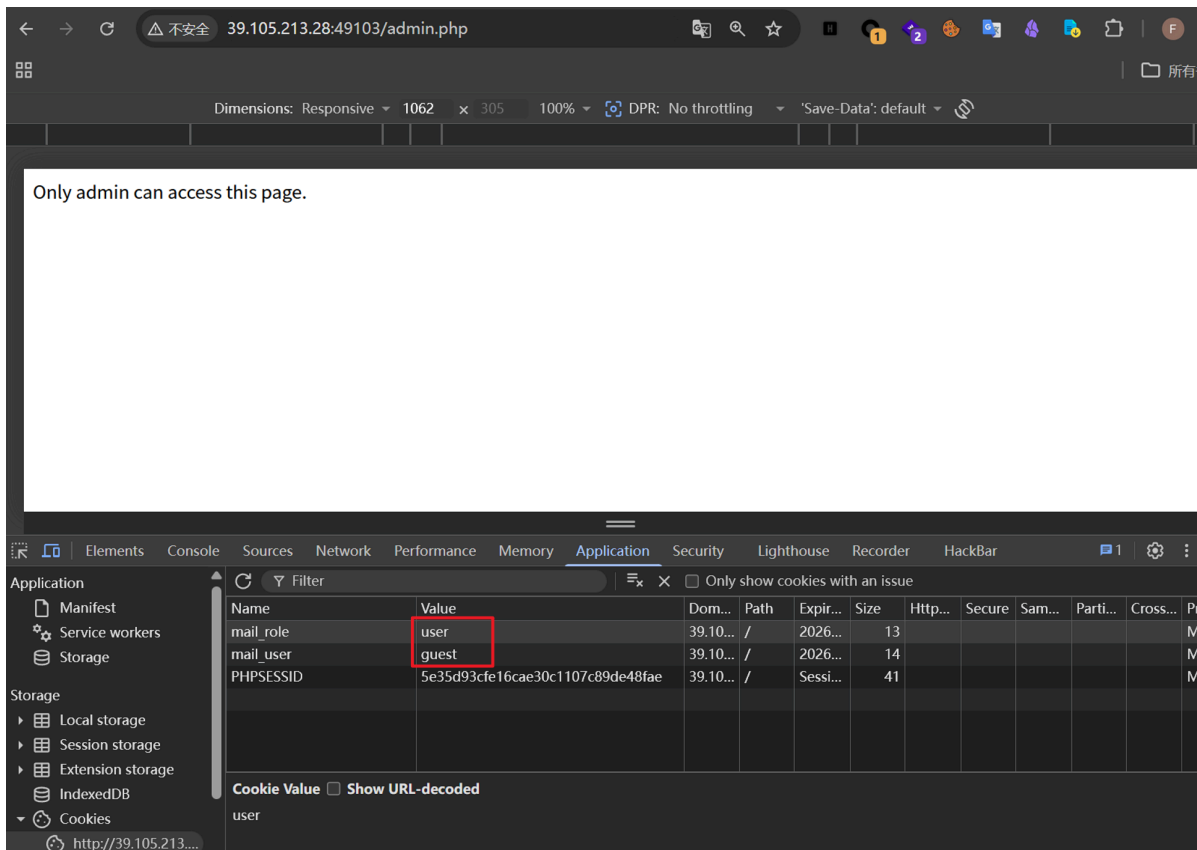
- 越权访问 (Broken Access Control)**：通过伪造明文Cookie提升权限至管理员。
- 逻辑缺陷与哈希碰撞**：利用PHP弱类型比较 (==) 造成MD5哈希碰撞，绕过双人复核机制。
- SSRF (服务端请求伪造)**：绕过内部路由限制，通过构造带URL编码的Payload读取本地 (127.0.0.1) 的Flag。

解题过程

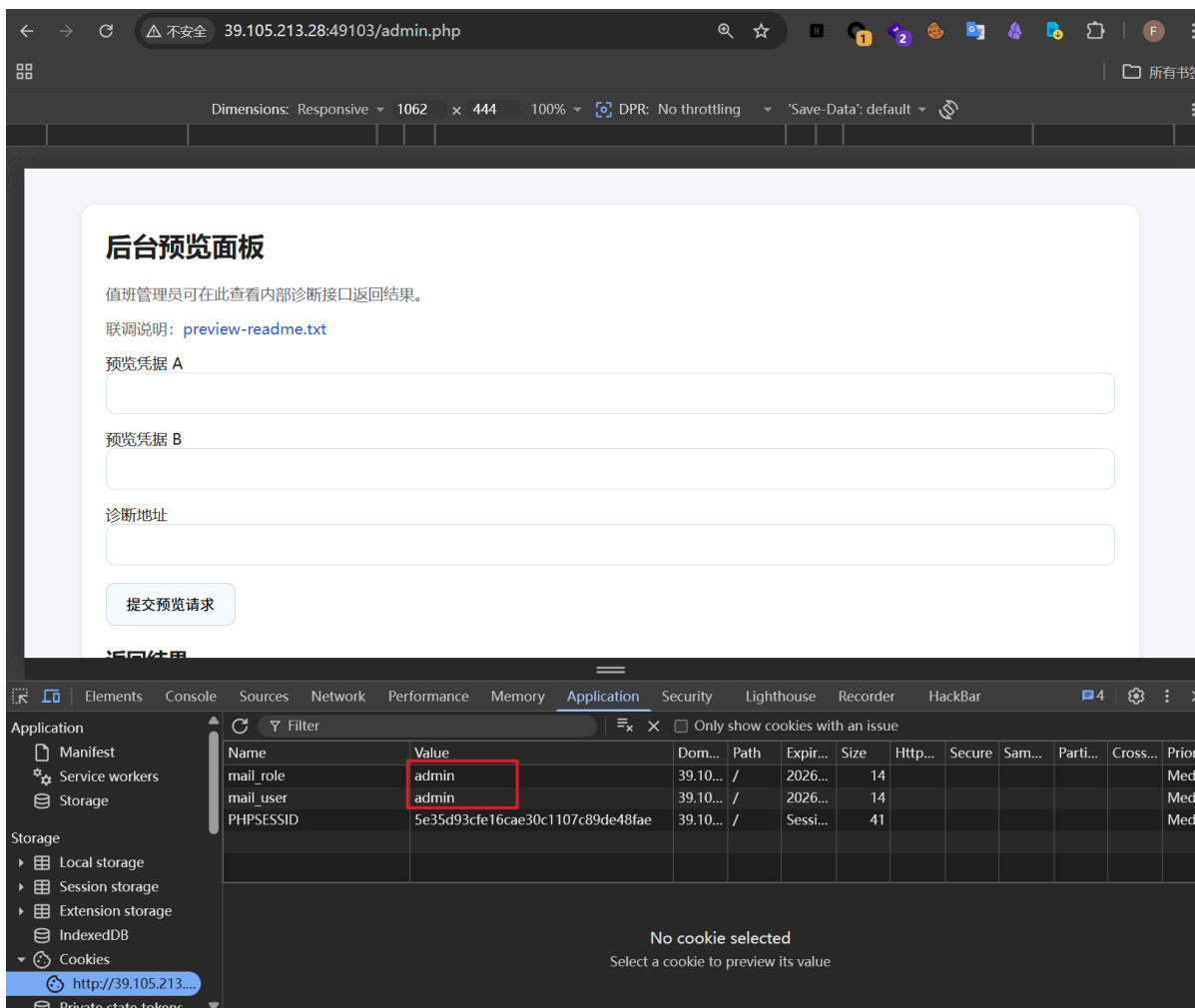
第一阶段：信息收集与Cookie伪造（权限验证绕过）

1.1 漏洞复现过程

- 信息收集**：通过阅读页面提供的测试邮件 (dev@simplemail.local)，发现提示：“前端页面会继续沿用浏览器侧那套旧标记，方便大家切身份联调”。这强烈暗示了系统存在**客户端信任 (Client-Side Trust) 漏洞**。
- 抓包与Cookie分析**：试图访问 /admin.php 抓包后，发现服务端返回的HTTP响应头中包含敏感的明文状态标记：Set-Cookie: mail_user=guest Set-Cookie: mail_role=user
- 利用方式**：直接在HTTP请求中修改Cookie值为 mail_user=admin; mail_role=admin，服务端没有在后端进行二次校验，直接信任了客户端传来的数据，导致成功访问后台预览面板。



F5刷新一下



1.2 深度知识点拓展（越权与防范机制）

- **代码层面的致命缺陷：** 产生此类漏洞的根本原因在于**后端直接信任了来自客户端可控的输入**。其脆弱的伪代码逻辑如下：

```
// 危险的代码示例 (vulnerable code)
if ($_COOKIE['mail_role'] == 'admin') {
    // 赋予管理员权限
}
```

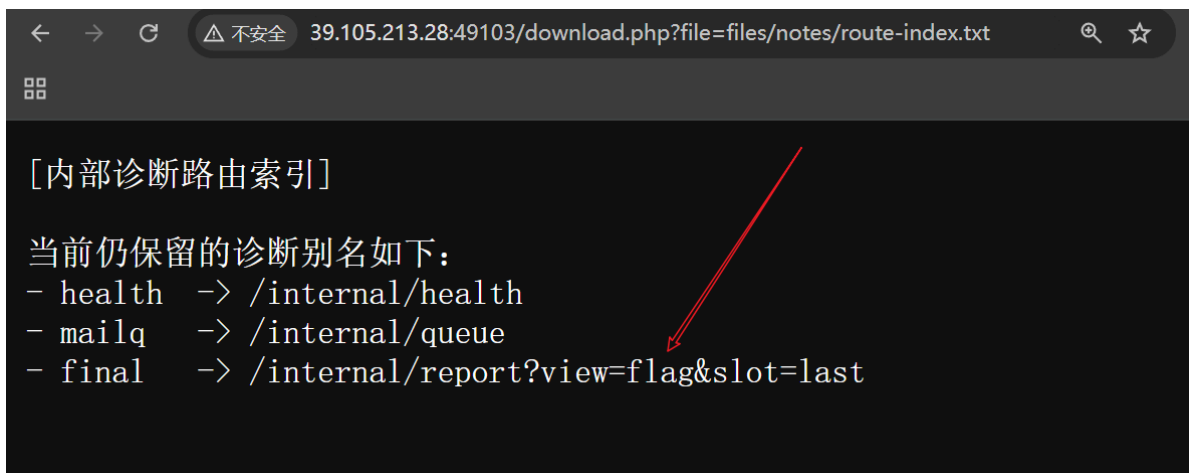
- **现代企业级防御方案（基于最新OWASP规范）：**

1. **服务端会话管理（Server-Side Sessions）：** 绝不将敏感身份标识明文存放在Cookie中。Cookie应仅存放无意义的、具有高熵值的 `Session ID`。具体的角色权限映射统一在服务器端的内存或数据库中维护。
2. **加密型令牌机制（如 JWT）：** 如果采用无状态的认证方式，必须对存放在客户端的身份信息进行**密码学签名**（如使用 HMAC 或 RSA）。若客户端篡改了 Payload（将user改为admin），由于缺乏服务端的密钥，其生成的签名将无法通过校验。
3. **安全属性加固：** 敏感Cookie必须设置 `HttpOnly`（防止客户端JS读取，缓解XSS风险）、`Secure`（仅允许HTTPS传输）以及 `SameSite=Strict/Lax`（防御CSRF跨站请求伪造攻击）。

查看联调说明：[preview-readme.txt](#)



访问/route-index.txt



抓取到了 Flag 的隐藏路径

第二阶段：逻辑缺陷与双人复核绕过（PHP弱类型哈希碰撞）

2.1 机制分析与突破口探寻



- **业务逻辑逆向：** 根据内部邮件提示：“先做摘要再比对，两份输入至少得看起来不是同一串”。说明后台执行了类似如下的逻辑：

```
if ($token_a != $token_b && md5($token_a) == md5($token_b)) {  
    // 复核通过  
}
```

- 接下来就是寻找两个不同的明文，使其MD5值在PHP的弱类型比较下相等。

2.2 漏洞原理：PHP 弱类型比较与 "0e" 魔法哈希 (Magic Hash)

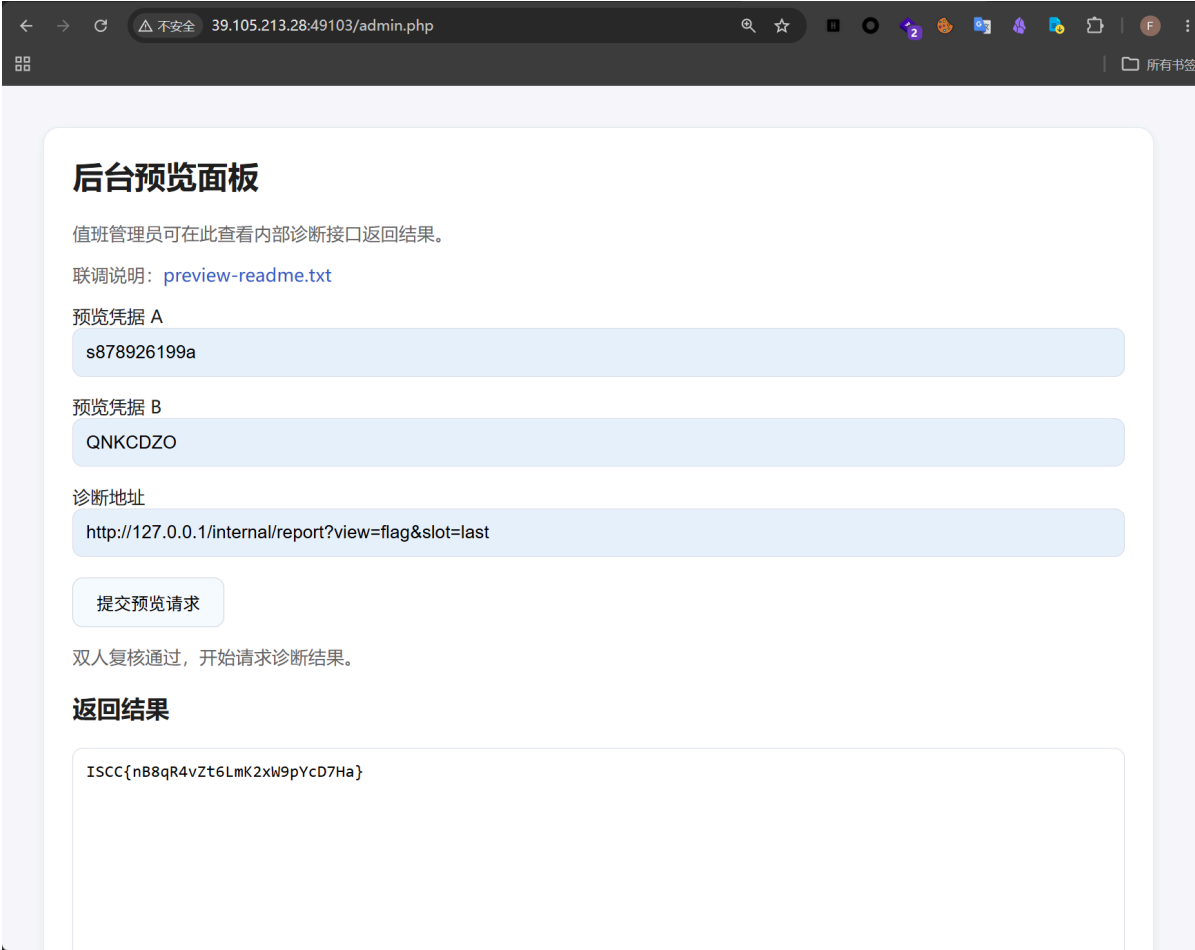
- **底层原因：** 在 PHP 中，当使用双等号 `==`（弱类型比较）来比对两个字符串时，如果这两个字符串都以 `"0e"` 开头，且后面跟着的都是数字，PHP 会非常“自作聪明”地把它们当作**科学计数法**来解析。
 - `0e` 代表 0×10^n 。
 - 无论后面的数字 `$n` 是多少，结果永远是 `$0$`。
 - 因此，PHP 会认为这两个字符串在数值上都等于 `$0$`，从而返回 `True`。
- **实战 Payload 解析：** WP 中给出的两个明文及其 MD5 结果如下：
 - `md5('s878926199a')` = `0e545993274517709034328855841020`
 - `md5('QNKCDZO')` = `0e830400451993494058024219903391`

虽然这两个明文完全不同（满足 `$token_a != $token_b`），且它们的真实 MD5 哈希值也完全不同，但因为它们的哈希值**恰好都以 0e 开头且后续全为纯数字**，触发了 PHP 的科学计数法解析漏洞，最终被判定为相等：`0 == 0`，成功绕过验证。

SSRF（服务端请求伪造）的本质，是我们控制服务器，让**服务器自己**去发起一个 HTTP 请求

我们需要明确告诉服务器去请求自己的 `127.0.0.1` 环回地址。目标URL：

`http://127.0.0.1/internal/report?view=flag&slot=last`



解题脚本

flag

```
ISCC{nB8qR4vZt6LmK2xw9pYcD7Ha}
```